RISC-V OPEN ARCHITECTURE PROJECT

Design & Verification of a 32-Bit RISC-V Single-Cycle Processor (RV32I)

A complete hardware implementation in Verilog HDL featuring modular datapath design, hierarchical control decoding, and cycle-accurate verification.

PRESENTER
Prince Raj

HARDWARE LANGUAGE
Verilog HDL (IEEE 1364)

SIMULATION TOOL
Icarus Verilog 13.0 & GTKWave

ARCHITECTURE
RV32I Base Integer ISA

EXECUTIVE SUMMARY

Project Objectives & Architecture Overview

Bridging theoretical computer architecture with synthesizable hardware implementation

🎯 Core Goals

- Implement a fully functional 32-bit single-cycle processor compliant with RISC-V RV32I specification.
- Design modular hardware blocks: PC, Memory, Register File, ALU, and Control Unit.
- Support R-type, I-type, Load, Store, and Branch instructions.

⚡ Why Single-Cycle?

- **CPI = 1**: Exactly one instruction completes per clock period.
- **Zero Hazard Complications**: No data forwarding or branch prediction stalls required.
- **Educational Foundation**: Provides the transparent baseline for pipelining and superscalar scaling.

🏆 Deliverables

- 8 synthesizable Verilog modules in `src/`.
- Top-level datapath integration (`riscv_core.v`).
- Comprehensive 11-instruction self-checking testbench.
- Interactive terminal simulation CLI (`run_interactive.py`).

INSTRUCTION SET ARCHITECTURE

RISC-V RV32I Instruction Formats

Regular, orthogonal 32-bit instruction layouts designed for efficient hardware decoding

Format	Bit [31:25]	Bit [24:20]	Bit [19:15]	Bit [14:12]	Bit [11:7]	Bit [6:0]	Typical Operations
R-Type	funct7 (7b)	rs2 (5b)	rs1 (5b)	funct3 (3b)	rd (5b)	opcode (7b)	`add, sub, and, or, xor, slt`
I-Type	immediate [11:0] (12b)		rs1 (5b)	funct3 (3b)	rd (5b)	opcode (7b)	`addi, andi, ori, lw, jalr`
S-Type	imm[11:5] (7b)	rs2 (5b)	rs1 (5b)	funct3 (3b)	imm[4:0] (5b)	opcode (7b)	`sw, sh, sb`
B-Type	imm[12|10:5]	rs2 (5b)	rs1 (5b)	funct3 (3b)	imm[4:1|11]	opcode (7b)	`beq, bne, blt, bge`

Key Architectural Features

- **Fixed 32-Bit Instruction Width**: Simplifies instruction fetch logic.
- **32 General Purpose Registers**: `x0` to `x31`, each 32 bits wide.
- **Hardwired Zero**: `x0` is physically tied to 0. Writing to it is ignored.

Hardware Decoding Advantages

- `rs1`, `rs2`, and `rd` always occupy the exact same bit positions across all formats.
- Opcode is always located in `bits [6:0]`, enabling instantaneous control unit decoding.

HARDWARE ORGANIZATION

The 5 Classic Stages of the Single-Cycle Datapath

All 5 phases execute concurrently in a continuous combinational flow during one clock period

STAGE 1

Fetch (IF)

- PC outputs current instruction address.
- Instruction Memory retrieves 32-bit machine word.
- Adder computes default `PC + 4`.

STAGE 2

Decode (ID)

- Control Unit decodes `opcode[6:0]`.
- Register File reads `rs1` and `rs2` data.
- Immediate Gen reconstructs 32-bit sign-extended constant.

STAGE 3

Execute (EX)

- ALU MUX selects Register 2 or Immediate.
- ALU performs arithmetic/logic computation.
- ALU calculates Zero flag and branch target (`PC + imm`).

STAGE 4

Memory (MEM)

- For `lw`: reads 32-bit word from Data RAM at ALU address.
- For `sw`: writes `rs2` data into RAM.
- Bypassed for R-type/I-type math.

STAGE 5

Write-Back (WB)

- Multiplexer selects Write-back source:
  - ALU result (Math)
  - Memory data (Load)
  - PC+4 (Jump link)
- Writes into register `rd`.

SUB-MODULE DESIGN

Modules 1 & 2: Program Counter & Instruction Memory

Instruction sequencing and memory-fetch hardware

Program Counter (`pc.v`)

The PC is a 32-bit sequential register. On each clock edge, it latches the next instruction address (`pc_next`).

```
always @(posedge clk or posedge rst) begin if (rst) begin pc <= 32'h0000_0000; // Reset to 0 end else begin pc <= pc_next; // Latch PC+4 or Branch end end
```

- **Non-blocking (<=)** avoids simulation race conditions.
- **Asynchronous reset** guarantees immediate initialization.

Instruction Memory (`instruction_mem.v`)

Word-indexed asynchronous ROM holding up to 64 instructions (256 bytes).

```
reg [31:0] memory [0:63]; // Asynchronous (instantaneous) read: assign instr = memory[pc[7:2]]; initial begin // Supports dynamic hex file loading: if (PROGRAM_FILE != "") $readmemh(PROGRAM_FILE, memory); end
```

- **Why `pc[7:2]`?** Instructions are 4-byte aligned (0, 4, 8, 12...). Slicing bits [7:2] divides byte address by 4 to get word index.

SUB-MODULE DESIGN

Module 3: 32x32-Bit Register File (reg_file.v)

High-speed dual-read, single-write internal register storage

Verilog Implementation

```
reg [31:0] registers [0:31]; // Dual Asynchronous Reads (x0 is hardwired to 0): assign rd_data1 = (rs1 == 5'd0) ? 32'b0 : registers[rs1]; assign rd_data2 = (rs2 == 5'd0) ? 32'b0 : registers[rs2]; // Synchronous Write on posedge clk: always @(posedge clk or posedge rst) begin if (rst) begin for (i = 0; i < 32; i = i + 1) registers[i] <= 32'b0; end else if (reg_write && (rd != 5'd0)) begin registers[rd] <= wr_data; end end
```

Critical Architectural Rules

- **2 Simultaneous Read Ports**: Reads operands for `rs1` and `rs2` combinationally in the Decode stage.
- **1 Synchronous Write Port**: Updates destination register `rd` only on the rising clock edge when `reg_write == 1`.
- **Hardwired Register x0**:
  - Reading `x0` always outputs `32'h00000000`.
  - Condition `(rd != 5'd0)` ensures writes to `x0` are silently dropped.
  - Enables compact RISC-V pseudo-instructions like `nop` (`addi x0, x0, 0`).

SUB-MODULE DESIGN

Module 4: Immediate Generator (imm_gen.v)

Assembling scrambled immediate fields and sign-extending to 32 bits

Why Immediate Generation is Needed

RISC-V intentionally scrambles immediate bit fields across instructions to keep register source and destination fields (`rs1`, `rs2`, `rd`) in fixed positions.

The Immediate Generator decodes the `opcode` and reconstructs the full 32-bit two's complement sign-extended number.

Sign Extension Magic:
`{{20{instr[31]}}, instr[31:20]}`
Replicates bit 31 (sign bit) 20 times to expand 12-bit signed numbers to 32 bits.

Format Decoding Table

```
case (opcode) // I-type (addi, lw): bits 31:20 7'b0010011, 7'b0000011:
imm_out = {{20{instr[31]}}, instr[31:20]}; // S-type (sw): split 31:25 and
11:7 7'b0100011: imm_out = {{20{instr[31]}}, instr[31:25], instr[11:7]};
// B-type (beq): bit 0 is hardwired to 0 7'b1100011: imm_out =
{{20{instr[31]}}, instr[7], instr[30:25], instr[11:8], 1'b0}; // J-type
(jal): 20-bit offset, bit 0 is 0 7'b1101111: imm_out = {{12{instr[31]}},
instr[19:12], instr[20], instr[30:21], 1'b0}; endcase
```

SUB-MODULE DESIGN

Module 5: ALU & Hierarchical ALU Control (alu.v)

High-speed arithmetic execution and multi-level control decoding

ALU Core (`alu.v`)

Takes two 32-bit operands (a, b) and performs the selected operation:

Control	Function	Verilog Expression
4'b0000	ADD	result = a + b;
4'b0001	SUB	result = a − b;
4'b0010	AND	result = a & b;
4'b0011	OR	result = a | b;
4'b0100	XOR	result = a ^ b;
4'b0101	SLT	result = ($signed(a) < $signed(b)) ? 1 : 0;

Zero Flag: `assign zero = (result == 32'b0);` (Crucial for branch decisions!)

Hierarchical ALU Decoder

Two-level decoding keeps the Main Control Unit fast and modular:

```
case (alu_op) 2'b00: alu_ctrl = ALU_ADD; // lw/sw: calculate address
2'b01: alu_ctrl = ALU_SUB; // beq: subtract rs1 - rs2 2'b10: // R-type
(checks funct3 and funct7[5]) case (funct3) 3'b000: alu_ctrl = funct7_5 ?
ALU_SUB : ALU_ADD; 3'b111: alu_ctrl = ALU_AND; 3'b110: alu_ctrl = ALU_OR;
3'b100: alu_ctrl = ALU_XOR; endcase 2'b11: // I-type arithmetic (addi,
andi, etc.) case (funct3) 3'b000: alu_ctrl = ALU_ADD; endcase endcase
```

SUB-MODULE DESIGN

Modules 6 & 7: Main Control Unit & Data Memory

Central traffic director and data memory access

Main Control Unit Truth Table (Decodes `opcode[6:0]`)

Instruction	Opcode	RegWrite	ALUSrc	MemRead	MemWrite	MemToReg	Branch	ALUOp
R-Type (add, sub)	`0110011`	1	0 (rs2)	0	0	00 (ALU)	0	10 (R-type)
I-Type (addi)	`0010011`	1	1 (imm)	0	0	00 (ALU)	0	11 (I-type)
Load (lw)	`0000011`	1	1 (imm)	1	0	01 (Mem)	0	00 (Add)
Store (sw)	`0100011`	0	1 (imm)	0	1	XX	0	00 (Add)
Branch (beq)	`1100011`	0	0 (rs2)	0	0	XX	1	01 (Sub)
Jump (jal)	`1101111`	1	X	0	0	10 (PC+4)	0	XX

Data Memory (`data_mem.v`)

- 64 words (256 bytes) RAM addressed by `addr[7:2]`.
- **Read**: Combinational on `mem_read == 1`.
- **Write**: Synchronous on `posedge clk` when `mem_write == 1`.

Key Architectural Safety

- Default case sets all control lines to 0.
- Prevents accidental memory corruption or unintended register overwrites on undefined opcodes.

SYSTEM INTEGRATION

Top Datapath Wiring: riscv_core.v

Connecting all 8 modules with multiplexers for unified execution

1. Next PC Multiplexer

Calculates whether to branch or step sequentially:

```
wire pc_branch_taken = (branch & alu_zero);
assign pc_next = (jump) ? branch_target :
(pc_branch_taken) ? branch_target :
pc_plus_4;
```

2. ALU Operand B MUX

Selects between Register 2 and Immediate constant:

```
assign alu_operand_b = (alu_src) ? imm_out :
rd_data2;
```

- 0: R-type math, BEQ comparison.
- 1: ADDI, LW/SW offset calculation.

3. Write-Back MUX

Selects data source written to register rd:

```
case (mem_to_reg) 2'b00: reg_wr_data =
alu_result; 2'b01: reg_wr_data = mem_rd_data;
2'b10: reg_wr_data = pc_plus_4; endcase
```

Routes ALU math, Memory load, or JAL return address.

EXECUTION WALKTHROUGH

Tracing an Instruction: ADD x3, x1, x2

Step-by-step signals when executing `0x002081B3` with x1=50, x2=30

Stage	Hardware Unit	Active Signals & Values	Hardware Action
1. Fetch	PC & IMEM	`PC = 0x00000008`	Instruction memory outputs `0x002081B3`. PC+4 is computed.
2. Decode	Control Unit & RegFile	`opcode=0110011, rs1=1, rs2=2, rd=3`	CU asserts `RegWrite=1`, `ALUSrc=0`. RegFile outputs `x1=50`, `x2=30`.
3. Execute	ALU & ALU Control	`alu_ctrl=4'b0000, A=50, B=30`	ALU performs addition: `50 + 30 = 80`. Zero flag = `0`.
4. Memory	Data Memory	`mem_read=0, mem_write=0`	Data memory is idle. ALU result passes directly to write-back MUX.
5. Write-Back	WB MUX & RegFile	`mem_to_reg=00, wr_data=80`	On rising clock edge, `80` is permanently written into register x3!

Contrast with Load Word (`lw x7, 4(x0)`)

For `lw`: ALU calculates address `0 + 4 = 4` $ightarrow$ Data Memory activates `mem_read=1` $ightarrow$ Memory reads data at address 4 (40) $ightarrow$ MUX selects `mem_to_reg=01` $ightarrow$ 40 is written to x7. This is the **longest path** in the entire CPU!

TESTING & VERIFICATION

Hardware Simulation & Self-Checking Verification

Automated testbench execution in Icarus Verilog verifying 11 assembly instructions

11-Instruction Test Program

PC	Instruction	Result
0x00	addi x1, x0, 15	x1 = 15
0x04	addi x2, x0, 25	x2 = 25
0x08	add x3, x1, x2	x3 = 40
0x0C	sub x4, x2, x1	x4 = 10
0x10	and x5, x1, x2	x5 = 9
0x14	or x6, x1, x2	x6 = 31
0x18	sw x3, 4(x0)	mem[4] = 40
0x1C	lw x7, 4(x0)	x7 = 40
0x20	beq x3, x7, 8	Branch Taken!
0x24	addi x8, x0, 99	Skipped! (x8=0)
0x28	addi x9, x0, 42	x9 = 42

Terminal Simulation Output

```
$ iverilog -o sim/riscv_sim src/*.v sim/tb_riscv.v $ vvp sim/riscv_sim
============================================================ FINAL REGISTER
FILE VALUES ============================================================ x0
(zero) = 0 (Expected: 0) x1 = 15 (Expected: 15) x2 = 25 (Expected: 25) x3
(add) = 40 (Expected: 40) x4 (sub) = 10 (Expected: 10) x5 (and) = 9
(Expected: 9) x6 (or) = 31 (Expected: 31) x7 (lw) = 40 (Expected: 40) x8
(skip) = 0 (Expected: 0 - skipped by branch!) x9 (target)= 42 (Expected:
42 - reached via branch!) Data Mem[4]= 40 (Expected: 40 - written by sw)
============================================================ TEST STATUS:
>>> SUCCESS! ALL INSTRUCTIONS VERIFIED <<<
============================================================
```

LIVE DEMONSTRATION

Interactive Terminal Demonstrator (`run_interactive.py`)

Live hardware demonstration utility for professors and presentations

⌨️ 1. User Input

Prompts user or professor for:

- ✦ Operation: `ADD, SUB, AND, OR, XOR, SLT`
- ✦ Operand 1 (loaded into x1)
- ✦ Operand 2 (loaded into x2)

Example command:

```
python3 run_interactive.py --op XOR --a 12 --b 5
```

⚙️ 2. Dynamic Assembly

• Assembles 32-bit binary machine codes on the fly.
• Generates `sim/instructions.hex`.
• Displays full bit-slice decomposition:
- `funct7` (31:25)
- `rs2` (24:20), `rs1` (19:15)
- `funct3` (14:12)
- `rd` (11:7), `opcode` (6:0)

📊 3. Hardware Telemetry

• Runs Icarus Verilog in real time.
• Prints cycle-by-cycle internal signals:
- ALU inputs and results
- Control unit states
- Write-back destination • Displays verified register results in hexadecimal and decimal.

PERFORMANCE ANALYSIS

Critical Path & Timing Analysis

Determining the maximum operating frequency of the single-cycle core

Clock Period Formulation

In a single-cycle processor, the clock cycle time ($ au_{clk}$) is constrained by the **longest delay** among all instructions:

```
$ au_{clk} \ge T_{PC} + T_{IMEM} + T_{REG\_RD} + T_{ALU} +
T_{DMEM} + T_{MUX} + T_{SETUP}$
```

Why Load Word (`lw`) sets the clock speed:
`lw` is the ONLY instruction requiring access to both instruction memory AND data memory in the same cycle. Every other instruction (e.g. `add`) finishes earlier but is forced to wait for the same clock period.

Single-Cycle vs Pipelined Comparison

Metric	Single-Cycle	5-Stage Pipelined
CPI	1.0	~1.1 - 1.3 (stalls)
Clock Speed	Slow (determined by `lw`)	Fast (~4-5x higher clock)
Hardware Complexity	Low (no hazard unit)	High (forwarding, stalls)
Throughput	1 instr / $ au_{long}$	~1 instr / $ au_{short}$

Single-cycle is ideal for teaching because dataflow is transparent with zero hidden state hazards.

VIVA VOCE PREPARATION

Professor Defense Guide: Top Q&A

Exact, authoritative answers to the most common viva questions

Q1: Why is register x0 hardwired to 0?

Answer: RISC-V eliminates dedicated hardware for operations like `nop` (implemented as `addi x0, x0, 0`) and `mv rd, rs` (implemented as `addi rd, rs, 0`). Hardwiring `x0=0` reduces instruction count and silicon area.

Q2: Why use non-blocking (<=) assignments?

Answer: Non-blocking assignments model physical flip-flops that sample their inputs simultaneously on the clock edge, preventing simulation race conditions.

Q3: How does branch decision work?

Answer: In the Execute stage, the ALU subtracts `rs1 - rs2`. If equal, the Zero flag is 1. The AND gate `(branch & zero)` triggers the MUX to select `PC + immediate` in the same cycle with zero penalty.

Q4: What is the purpose of ALU Control?

Answer: Hierarchical decoding separates the 7-bit opcode decode from the `funct3/funct7` field decode. This prevents a giant monolithic decoder, reducing combinational gate delay.

Q5: Why is memory addressed as `pc[7:2]`?

Answer: Memory in RISC-V is byte-addressed, but each instruction is 4 bytes (32 bits). Slicing `pc[7:2]` divides by 4, mapping byte addresses 0, 4, 8, 12... to word slots 0, 1, 2, 3...

Q6: What is the critical path?

Answer: The Load Word (`lw`) instruction. It traverses PC $ ightarrow$ IMEM $ ightarrow$ RegFile Read $ ightarrow$ ALU $ ightarrow$ DMEM $ ightarrow$ WB MUX $ ightarrow$ RegFile Write.

SUMMARY & FUTURE SCOPE

Conclusion & Future Roadmap

Summary of outcomes and architectural roadmap for future research

🏆 Accomplishments

- Engineered 8 modular, synthesizable Verilog modules conforming to RV32I.
- Integrated full datapath with multi-source multiplexing and hierarchical control.
- 100% verified against arithmetic, logic, memory, and branch operations.
- Built live interactive CLI demonstrator for real-time validation.

🚀 Next Steps: Pipelining

- Introduce 4 pipeline register stages: `IF/ID`, `ID/EX`, `EX/MEM`, `MEM/WB`.
- Implement Hazard Detection Unit to resolve data and control hazards.
- Implement Forwarding Unit (ALU-to-ALU bypass) to eliminate stalls.

⚡ ISA Extensions

- **RV32M**: Hardware multiplier & divider blocks (`mul`, `div`, `rem`).
- **Memory Hierarchy**: Direct-mapped or 2-way set-associative L1 instruction/data caches.
- **Interrupts & CSRs**: Control and Status Registers for timer interrupts and system calls.